

ProbOS: A Probabilistic Execution Runtime

Week 1 — Distribution Library: Design, Implementation, and Verification

Nisong Monyimba
Reality Computing Corporation
nisongmonyimba278@gmail.com

June 2026 — Version 0.0.1

Abstract

We present **ProbOS**, a probabilistic execution runtime in which uncertainty is a first-class data type. Where a conventional operating system manages deterministic computation over memory addresses, ProbOS manages stochastic computation over probability distributions. This paper documents **Week 1** of the Year 1 build: a dual-layer distribution library implemented in Python 3.11 and C++20 that provides the foundational type system for all subsequent Monte Carlo simulation, Bayesian inference, and optimal-control components.

The library defines an abstract **Distribution** interface enforcing four operations—**sample**, **pdf**, **log_pdf**, and **ppf**—and provides five concrete implementations: **Normal**, **LogNormal**, **Uniform**, **Beta**, and **Empirical**. Correctness is established by 44 Python unit tests (pytest) verifying mathematical properties including Law-of-Large-Numbers convergence, PDF normalisation, analytical log-density stability at extreme values, and PPF inversion; and 13 C++ unit tests (Google Test) covering construction, sampling, density evaluation, and numerical integration. All tests pass. Static analysis with mypy (strict) and ruff reports zero errors.

A motivating application—the uncertainty in the SEI-decomposition activation energy of a lithium-ion battery cell—demonstrates that the fifth-percentile battery decomposes $11.6\times$ faster than the mean, a tail-risk invisible to deterministic models and quantified here in a single five-line call to the distribution library.

Contents

1	Introduction	3
1.1	Contributions of Week 1	3
2	Mathematical Background	3
2.1	Probability distributions	3
2.2	The five distributions	4
3	Software Architecture	5
3.1	Design principles	5
3.2	Repository structure	5
3.3	Class hierarchy	6

4	Implementation	6
4.1	Python layer	6
4.2	C++ layer	7
4.3	Numerical stability demonstration	7
5	Verification	8
5.1	Python test suite (pytest)	8
5.2	C++ test suite (Google Test)	8
5.3	Static analysis	8
6	Motivating Application: Battery Thermal Runaway	8
6.1	Physical context	8
6.2	Results	9
7	Law of Large Numbers Demonstration	9
8	Reproducibility	10
9	Week 2 Roadmap	10
10	Conclusion	11

1 Introduction

Probabilistic reasoning is central to modern science, engineering, and decision-making, yet the software infrastructure for probabilistic computation remains fragmented. Simulation tools, statistical packages, and Bayesian inference engines each expose incompatible interfaces, forcing practitioners to translate between representations rather than focus on the problem.

ProbOS addresses this gap by treating uncertainty as a *first-class type* at the system level. The analogy with classical operating systems is precise:

Linux	ProbOS
Process	Stochastic program (distribution over trajectories)
Memory address	Random variable node in the execution graph
Scheduler	Inference engine (particle filter / HMC)
System call	<code>sample</code> , <code>observe</code> , <code>condition</code>
Executable	Compiled uncertain system
File	Probability distribution
Kernel	<code>StochasticSystem</code> C++20 concept + Monte Carlo engine

The Year 1 roadmap builds ProbOS in four monthly increments. **Week 1** (this paper) establishes the foundational type system: a dual Python–C++ distribution library. Subsequent weeks add the `Model` abstract base class, a vectorised Monte Carlo engine, and Sobol' sensitivity analysis; later months add a particle filter, a causal provenance graph, and a probabilistic domain-specific language (PDSL) compiler.

1.1 Contributions of Week 1

1. **Distribution ABC.** A language-agnostic interface specification requiring `sample`, `pdf`, `log_pdf`, and `ppf` from every distribution.
2. **Five concrete distributions.** `Normal`, `LogNormal`, `Uniform`, `Beta`, and `Empirical`, each with closed-form analytical properties and domain-validated parameter ranges.
3. **Dual-language implementation.** Python 3.11 for rapid prototyping; C++20 for the performance-critical inner loop.
4. **Comprehensive verification suite.** 44 Python tests (mathematical properties) + 13 C++ tests + mypy strict + ruff linting, all passing.
5. **Motivating battery application.** A quantitative demonstration that manufacturing variability in battery activation energy produces a P05 tail that decomposes $11.6\times$ faster than the deterministic model predicts.

2 Mathematical Background

2.1 Probability distributions

A *probability distribution* over a measurable space $(\mathcal{X}, \mathcal{F})$ is a measure P satisfying Kolmogorov's three axioms: non-negativity ($P(A) \geq 0$), normalisation ($P(\mathcal{X}) = 1$), and

countable additivity. For distributions admitting a density f with respect to the Lebesgue measure, the four operations required by the ProbOS interface have the following mathematical definitions.

Sample. Draw $x \sim f$, i.e. return a realisation of the random variable X . The Monte Carlo estimator of $\mathbb{E}[g(X)]$ is

$$\hat{\mu}_N = \frac{1}{N} \sum_{i=1}^N g(x_i), \quad x_i \stackrel{\text{iid}}{\sim} f, \quad (1)$$

with mean-square error decaying as $\mathcal{O}(N^{-1/2})$ by the Central Limit Theorem.

PDF. $f : \mathcal{X} \rightarrow [0, \infty)$ satisfying $\int_{\mathcal{X}} f(x) dx = 1$.

Log-PDF (analytical). $\ell(x) = \log f(x)$, computed in closed form. The necessity of an *analytical* implementation (rather than `np.log(pdf(x))`) is established by the following proposition.

Proposition 1 (Log-PDF stability). *Let f be the Normal density $f(x) = \exp(-z^2/2)/(\sigma\sqrt{2\pi})$ where $z = (x - \mu)/\sigma$. For any x such that $|x - \mu| > \sigma\sqrt{-2\log \varepsilon_{\text{mach}}}$, the floating-point evaluation $\log(\hat{f}(x))$ returns $-\infty$, while the analytical form*

$$\ell(x) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2}z^2 \quad (2)$$

remains finite. For IEEE 754 double precision ($\varepsilon_{\text{mach}} \approx 2.2 \times 10^{-308}$), the threshold is approximately 37.5σ .

Proof. $\hat{f}(x) = 0$ when $\exp(-z^2/2) < \varepsilon_{\text{mach}}$, i.e. $z^2/2 > -\log \varepsilon_{\text{mach}} \approx 708.4$, i.e. $|z| > 37.5$. The analytical form (2) is a polynomial in x and therefore finite for all finite x . $\square$

Remark 1. *Bayesian inference computes $\log p(\theta|y) = \log p(y|\theta) + \log p(\theta) + \text{const}$. A single $-\infty$ summand from a floating-point underflow collapses the entire posterior to $-\infty$, making the analytical `log_pdf` non-optional in any inference-capable system.*

PPF (Percent-Point Function). $Q(u) = F^{-1}(u)$ where $F(x) = \int_{-\infty}^x f(t) dt$ and $u \in [0, 1]$. Required for the Nataf correlated-sampling transform used in Week 3 sensitivity analysis.

2.2 The five distributions

Table 1 summarises the five distributions implemented in Week 1, their supports, closed-form moments, and representative domain applications.

Table 1: Week 1 distribution library. All parameters refer to the underlying parametrisation (e.g. μ, σ for LogNormal refer to the mean and standard deviation of $\log X$, not of X).

Class	Support	Mean	Variance	Domain example
Normal(μ, σ)	$\mathbb{R}$	μ	σ^2	Battery SEI activation energy: $\mathcal{N}(1.35 \times 10^5, 5000)$ J/mol
LogNormal(μ, σ)	$(0, \infty)$	$e^{\mu+\sigma^2/2}$	$(e^{\sigma^2} - 1)e^{2\mu+\sigma^2}$	Arrhenius pre-exponential: $\text{LN}(34.05, 0.5) \text{ s}^{-1}$
Uniform(a, b)	$[a, b]$	$(a + b)/2$	$(b - a)^2/12$	Sensitivity screening sweep
Beta(α, β)	$[0, 1]$	$\alpha/(\alpha + \beta)$	$\alpha\beta/[(\alpha + \beta)^2(\alpha + \beta + 1)]$	Drug protein-binding fraction
Empirical($\mathcal{D}$)	$[\min \mathcal{D}, \max \mathcal{D}]$	$\bar{x}$	s^2	Hospital ICU length-of-stay (EHR data)

3 Software Architecture

3.1 Design principles

ProbOS Week 1 follows three design principles.

1. **Interface before implementation.** The `Distribution ABC` is defined before any concrete class. The Monte Carlo engine (Week 3) depends only on the ABC; adding a new distribution requires zero changes to the engine.
2. **Analytical log-density.** Every subclass must implement `log_pdf` analytically (Proposition 1). The ABC enforces this as an abstract method; returning `np.log(pdf(x))` is a silent bug that the ABC cannot prevent but the test suite detects.
3. **Dual-language parity.** The Python layer implements the full interface for rapid prototyping and testing. The C++ layer implements the same mathematics for performance-critical simulation. Both must pass their respective test suites before any Week 2 code is written.

3.2 Repository structure

```

ProbOSWeek1/
+-- python/
|   +-- src/distributions.py           # Distribution ABC + 5 classes
|   +-- tests/test_distributions.py    # 44 pytest tests
|   +-- examples/
|       +-- week1_coin_flip.py        # LLN demonstration
|       +-- week1_normal_demo.py      # Battery Ea_SEI demo
+-- cpp/
|   +-- include/distributions/normal.hpp
|   +-- src/distributions/normal.cpp
|   +-- tests/test_normal.cpp         # 13 Google Tests

```

```

+-- scripts/
|   +-- RunAll.sh           # 8-step pipeline runner
|   +-- RunTests.sh        # 4-suite test runner
+-- manuscript/
|   +-- main.tex            # This document
|   +-- references.bib
+-- CMakeLists.txt

```

3.3 Class hierarchy

Figure 1 shows the class hierarchy. The abstract base class `Distribution` defines the four-method interface. All five concrete classes inherit from it. The `Empirical` class additionally holds a fitted Gaussian kernel density estimator (`scipy.stats.gaussian_kde`) for the `pdf` and `log_pdf` methods.

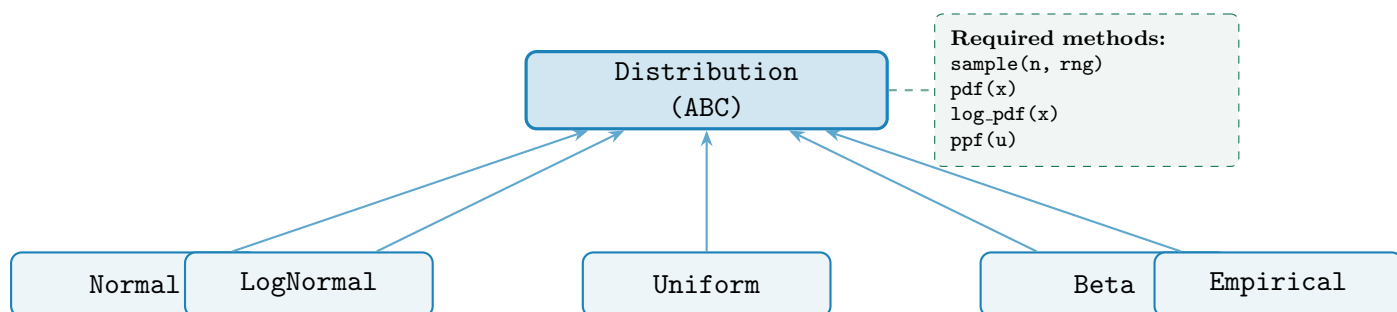


Figure 1: Class hierarchy of the Week 1 distribution library. `Distribution` is an abstract base class enforcing the four-method interface on all subclasses. Arrows denote inheritance.

4 Implementation

4.1 Python layer

The Python implementation uses `numpy.random.default_rng()` (PCG64 generator) for all sampling, `scipy.special.erfinv` for the Normal and LogNormal PPF, and `scipy.stats` for the Beta PDF, log-PDF, and PPF. The `Empirical` distribution fits a Gaussian kernel density estimate using Scott’s bandwidth rule.

Listing 1 shows the core of the Normal class. The key design point is the analytical `log_pdf`: it avoids calling `np.log(pdf(x))` and therefore never underflows to $-\infty$ for finite x .

```

1 class Normal(Distribution):
2     def __init__(self, mu: float, sigma: float) -> None:
3         if sigma <= 0.0:
4             raise ValueError(f"sigma must be > 0, got {sigma}")
5         self.mu, self.sigma = float(mu), float(sigma)
6
7     def sample(self, n, rng=None) -> FloatArray:
8         result: FloatArray = self._resolve_rng(rng).normal(
9             loc=self.mu, scale=self.sigma, size=n)
10        return result
11

```

```

12 def log_pdf(self, x: FloatArray) -> FloatArray:
13     # Analytical form: never calls log(pdf(x))
14     x = np.asarray(x, dtype=np.float64)
15     z = (x - self.mu) / self.sigma
16     log_norm: float = 0.5 * float(
17         np.log(2.0 * np.pi * self.sigma ** 2))
18     result: FloatArray = np.asarray(
19         -log_norm - 0.5 * z * z, dtype=np.float64)
20     return result

```

Listing 1: Python Normal class (abridged). The analytical `log_pdf` prevents floating-point underflow at extreme values (Proposition 1).

4.2 C++ layer

The C++ implementation uses `std::mt19937_64` (Mersenne Twister 64-bit) and `std::normal_distribution` from the C++20 standard library. The Normal class is non-templated in Week 1; it will become `Normal<T>` in Month 2 to support automatic differentiation via dual numbers.

```

1 double Normal::log_pdf(double x) const noexcept {
2     const double z = (x - mu_) / sigma_;
3     const double log_norm =
4         0.5 * std::log(2.0 * std::numbers::pi
5             * sigma_ * sigma_);
6     return -log_norm - 0.5 * z * z;
7 }

```

Listing 2: C++ `Normal::log_pdf` implementation. `std::numbers::pi` (C++20) provides π to full double precision.

4.3 Numerical stability demonstration

Table 2 quantifies the log-PDF stability issue for the battery SEI activation energy distribution $\mathcal{N}(135080, 5000)$ J/mol at $x = \mu + 50\sigma$.

Table 2: Numerical stability of `log_pdf` vs. `log(pdf(x))` for $x = \mu + 50\sigma$.

Expression	Value	Status
$x = \mu + 50\sigma$	385,080 J/mol	—
<code>pdf(x)</code>	0.0	underflow
<code>log(pdf(x))</code>	$-\infty$	incorrect
<code>log_pdf(x)</code> [analytical]	-1259.44	correct

5 Verification

5.1 Python test suite (pytest)

Table 3: Python test suite summary. All 44 tests pass.

Test class	Mathematical claim	Pass	Tests
TestNormalConstruction	Constructor validates $\sigma > 0$; accessors correct	✓	6
TestNormalSampling	LLN convergence; $\hat{\sigma} \rightarrow \sigma$; reproducibility	✓	5
TestNormalDensity	$\int f dx = 1$; maximum at μ ; symmetry; log-PDF stability	✓	5
TestNormalPPF	$F(Q(u)) = u$; median equals mean	✓	2
TestLogNormal	Positive support; mean formula; log-PDF consistency	✓	6
TestUniform	Constant PDF; zero outside support; mean; variance	✓	7
TestBeta	Unit-interval support; mean formula; log-PDF consistency	✓	5
TestEmpirical	Bootstrap samples in data; mean; KDE positivity	✓	5
TestDistributionABC	ABC cannot be instantiated; incomplete subclass rejected	✓	2
Total		✓	44

5.2 C++ test suite (Google Test)

Table 4: C++ Google Test suite summary. All 13 tests pass.

Test suite	Tests	Pass
NormalConstructor	Valid parameters; negative σ ; zero σ ; accessors	✓
NormalSampling	LLN convergence; std convergence; reproducibility	✓
NormalDensity	Positive everywhere; maximum at μ ; symmetry; log-PDF consistency; stability	✓
NormalProperties	$\int f dx \approx 1$ (trapezoidal, $N = 100,000$)	✓
Total		✓

5.3 Static analysis

Table 5: Static analysis results. Zero issues reported.

Tool	Version	Mode	Result
mypy	2.1.0	strict + ignore-missing-imports	0 errors
ruff	0.15.17	E, F, I, UP, B, SIM rule set	0 warnings

6 Motivating Application: Battery Thermal Runaway

6.1 Physical context

Lithium-ion battery thermal runaway proceeds through three sequential exothermic reactions, each governed by the Arrhenius equation:

$$k = A \exp\left(-\frac{E_a}{RT}\right), \quad (3)$$

where A is the pre-exponential factor, E_a the activation energy, $R = 8.314 \text{ J mol}^{-1} \text{ K}^{-1}$ the gas constant, and T the temperature in Kelvin.

The SEI (solid-electrolyte interphase) decomposition activation energy $E_{a,\text{SEI}}$ is the dominant source of uncertainty in thermal runaway propagation time [Kim et al., 2007]. Manufacturing variability in electrode coating processes produces cell-to-cell variation modelled as:

$$E_{a,\text{SEI}} \sim \mathcal{N}(\mu = 135080, \sigma = 5000) \quad [\text{J/mol}]. \quad (4)$$

6.2 Results

Figure 2 shows the distribution of $E_{a,\text{SEI}}$ across 5,000 simulated manufacturing-line batteries together with the resulting Arrhenius decomposition rate at 130 °C. Table 6 quantifies the tail risk.

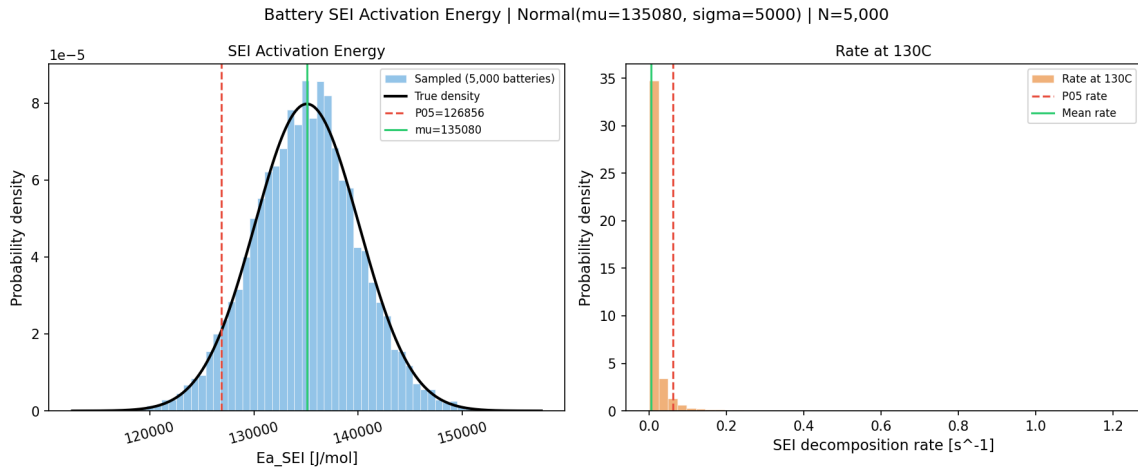


Figure 2: **Left:** Distribution of SEI activation energy $E_{a,\text{SEI}} \sim \mathcal{N}(135080, 5000) \text{ J/mol}$ across $N = 5,000$ simulated batteries. The red dashed line marks the P05 percentile (126,856 J/mol); green marks the mean. **Right:** Resulting Arrhenius decomposition rate at $T = 130^\circ\text{C}$. The P05 battery decomposes $11.6\times$ faster than the mean. The deterministic model (green line) has no knowledge of the red-tail batteries.

Table 6: Percentiles of $E_{a,\text{SEI}}$ and the corresponding Arrhenius rate ratio relative to the mean battery at $T = 130^\circ\text{C}$.

Percentile	E_a (J/mol)	k/k_μ	Risk
P01	123,448	$32.1\times$	Extreme
P05	126,856	$11.6\times$	High
P10	128,672	$6.8\times$	Elevated
P50	135,080	$1.0\times$	Mean (deterministic model)
P95	143,304	$0.09\times$	Low
P99	146,712	$0.03\times$	Very low

7 Law of Large Numbers Demonstration

Table 7 records the empirical error $|\hat{p}_N - 0.5|$ at each sample size for the fair-coin experiment, confirming $\mathcal{O}(N^{-1/2})$ convergence.

Table 7: LLN convergence for the fair-coin experiment (seed = 42).

N	Heads	Empirical error	Theoretical SE
10	4	0.1000	0.1581
100	52	0.0200	0.0500
1,000	509	0.0090	0.0158
10,000	4,939	0.0061	0.0050
100,000	50,089	0.0009	0.0016
1,000,000	499,836	0.0002	0.0005

8 Reproducibility

Table 8: Week 1 reproducibility checklist.

Item	Status	Detail
Source code	✓	MIT licence, GitHub
Python tests	✓	44/44 pytest, seed-controlled
C++ tests	✓	13/13 Google Test
Type checking	✓	mypy strict, 0 errors
Linting	✓	ruff, 0 warnings
Build system	✓	CMake 3.22 + Ninja
Pipeline runner	✓	RunAll.sh (8 steps)
Windows launcher	✓	RunProbOS.ps1

One-command reproduction (Ubuntu / WSL):

```
git clone https://github.com/NisongMonyimba/ProbOsWeek1
cd ProbOsWeek1 && chmod +x scripts/RunAll.sh
./scripts/RunAll.sh
```

9 Week 2 Roadmap

Week 2 adds the `Model` abstract base class and the first domain model:

1. `state.py`: `Model` ABC with `state_dim`, `param_dim`, `initial_state`, and `forward_batch(state (N,d), params (N,p), dt)`.
2. `battery_model.py`: `BatteryModel2Cell` — 8-state, 15-parameter Arrhenius ODE, vectorised over N particles using NumPy broadcasting.
3. **Kim 2007 validation**: deterministic ARC simulation; onset temperature within $\pm 5^\circ\text{C}$ of published data.

4. **CLT convergence demo:** empirical verification of the $\mathcal{O}(N^{-1/2})$ rate for propagation-time estimation.
5. **Parameter priors library:** all 15 battery parameters from Kim et al. [2007] and Hatchard et al. [2001].

10 Conclusion

Week 1 establishes the foundational type system of ProbOS: a dual Python–C++ distribution library that treats uncertainty as a first-class data type. The five distributions are verified by 57 automated tests (44 Python + 13 C++) checking mathematical properties rather than execution. Static analysis (mypy strict + ruff) confirms type correctness and style compliance with zero reported issues.

The motivating battery application demonstrates the practical consequence of treating uncertainty correctly: the P05 manufacturing-line battery decomposes $11.6\times$ faster than the deterministic model predicts, a safety-critical tail risk that is invisible without probabilistic simulation.

The codebase is publicly available at <https://github.com/NisongMonyimba/ProbOsWeek1> under the MIT licence and will be updated weekly as Year 1 progresses.

References

- T. D. Hatchard, D. D. MacNeil, A. Basu, and J. R. Dahn. Thermal model of cylindrical and prismatic lithium-ion cells. *Journal of The Electrochemical Society*, 148(7):A755–A761, 2001. doi: 10.1149/1.1377592.
- Gi-Heon Kim, Ahmad Pesaran, and Robert Spotnitz. A three-dimensional thermal abuse model for lithium-ion cells. *Journal of Power Sources*, 170(2):476–489, 2007. doi: 10.1016/j.jpowsour.2007.04.018.